Ndless Dev Guide — TI-Nspire CX CAS II-T

> Guide de développement C pour Ndless sur TI-Nspire CX CAS II-T.
> Libs couvertes : libndls · os.h (libc) · nGC · nGL · SDL 1.2 · freetype · zlib
> Toolchain : Docker `ndless-sdk` · `nspire-gcc` · `nspire-ld` · `genzehn` · `make-prg`

---

1. La calc et son hardware

Propriété	Valeur
CPU	ARM Cortex-A9 (ARMv7)
Fréquence	~396 MHz
RAM	~64 MB
Écran	320×240 px, RGB565 (16 bits/pixel)
Orientation écran	Paysage (320 large, 240 haut)
Stockage	Flash NAND interne
OS	TI-Nspire OS (Nucleus RTOS)

La CX CAS II-T est une **HW-W** — son écran natif est `SCR_240x320_565` (rotation logicielle), mais en pratique on travaille toujours en 320×240 avec les libs.

---

2. Toolchain & compilation

Structure d'un projet

```
MonProjet/
├── Makefile
├── main.c
└── (autres .c)
```

Makefile de base

```
BINARY  = MonProjet
CFLAGS  = -Wall -W -marm -Os -I/opt/ndless-libs/include
LDFLAGS = -L/opt/ndless-libs/lib
LIBS    =
OBJS    = main.o

all: $(BINARY).tns

%.o: %.c
    nspire-gcc $(CFLAGS) -c $< -o $@

$(BINARY).elf: $(OBJS)
    nspire-ld $(LDFLAGS) $(OBJS) $(LIBS) -o $@

$(BINARY).tns: $(BINARY).elf
    genzehn --input $< --output $(BINARY).tns.zehn --name "$(BINARY)" --2
    make-prg $(BINARY).tns.zehn $(BINARY).tns
    rm -f $(BINARY).tns.zehn $(BINARY).elf

clean:
    rm -f *.o *.elf *.tns *.zehn
```

Ajouter des libs au Makefile

```
LIBS = -lSDL          # SDL 1.2
LIBS = -lngl          # nGL
LIBS = -lfreetype     # FreeType
LIBS = -lz            # zlib
LIBS = -lSDL -lngl    # plusieurs libs
```

Flag genzehn important

`--240x320-support true` → **obligatoire sur CX CAS II-T**. Sans ça, Ndless lance le programme en mode compatibilité vieilles calcs → freeze + mauvaise résolution.

Commandes utiles

```
# Compiler
make
```

```
# Tout recompiler depuis zéro
make clean && make

# Lancer le manager de projet custom
python3 ~/Documents/Ndless/n-dev.py
```

---

3. Squelette de programme

```
#include <libndls.h>
#include <os.h>

int main(void) {
    // Ton code ici

    while (!any_key_pressed()) {}  // Attendre une touche avant de quitte

    return 0;
}
```

> `return 0` quitte proprement et retourne au menu Ndless.

---

4. libndls — Référence complète

Header : `#include <libndls.h>`
Linkée automatiquement, rien à ajouter dans `LIBS` .

4.1 LCD API

```
scr_type_t lcd_type();
```

Retourne le type d'écran natif de la calc. Sur CX CAS II-T → `SCR_240x320_565` .

```
bool lcd_init(scr_type_t type);
```

Initialise le mode LCD. À appeler avant `lcd_blit` . Appeler `lcd_init(SCR_TYPE_INVALID)` avant d'utiliser les fonctions UI et avant de quitter.

```
void lcd_blit(void* buffer, scr_type_t type);
```

Envoie un buffer à l'écran.

Types d'écran disponibles :

Constante	Description
`SCR_320x240_565`	RGB565, natif sur CX (avant HW-W)
`SCR_240x320_565`	RGB565, natif sur CX HW-W (ta calc)
`SCR_320x240_4`	4-bit grayscale, vieilles calcs
`SCR_320x240_8`	8-bit palette
`SCR_320x240_16`	RGB444

Exemple — framebuffer direct :

```
#include <libndls.h>
#include <os.h>
#include <string.h>

#define W 320
#define H 240

int main(void) {
    uint16_t buffer[W * H];

    lcd_init(SCR_320x240_565);

    // Remplir en rouge (RGB565 : R=31, G=0, B=0 → 0xF800)
    for (int i = 0; i < W * H; i++)
        buffer[i] = 0xF800;

    lcd_blit(buffer, SCR_320x240_565);

    while (!any_key_pressed()) {}

    lcd_init(SCR_TYPE_INVALID);  // Remettre l'écran en mode OS
    return 0;
}
```

RGB565 — comment encoder une couleur :

```
// Format : RRRRRGGGGGGBBBBB (5 bits R, 6 bits G, 5 bits B)
// R, G, B sont dans [0..255], on les convertit :
uint16_t rgb565(uint8_t r, uint8_t g, uint8_t b) {
    return ((r >> 3) << 11) | ((g >> 2) << 5) | (b >> 3);
}

// Exemples :
uint16_t rouge  = 0xF800;  // (255, 0, 0)
uint16_t vert   = 0x07E0;  // (0, 255, 0)
uint16_t bleu   = 0x001F;  // (0, 0, 255)
uint16_t blanc  = 0xFFFF;
uint16_t noir   = 0x0000;
uint16_t jaune  = 0xFFE0;  // (255, 255, 0)
uint16_t cyan   = 0x07FF;
uint16_t magenta = 0xF81F;
```

Dessiner un pixel :

```
buffer[y * W + x] = couleur;
```

Dessiner un rectangle :

```
void draw_rect(uint16_t *buf, int x, int y, int w, int h, uint16_t color)
    for (int row = y; row < y + h; row++)
        for (int col = x; col < x + w; col++)
            buf[row * 320 + col] = color;
}
```

4.2 UI — Popups

```
void show_msgbox(const char *title, const char *msg);
```

Affiche une popup avec un bouton OK.

```
// Exemple
show_msgbox("Erreur", "Fichier introuvable !");
```

```
unsigned int show_msgbox_2b(const char *title, const char *msg,
                            const char *btn1, const char *btn2);
```

Popup avec 2 boutons. Retourne `1` si btn1 pressé, `2` si btn2.

```
// Exemple
unsigned int choix = show_msgbox_2b("Quitter ?", "Tu veux vraiment quitte
if (choix == 1) return 0;
```

```
unsigned int show_msgbox_3b(const char *title, const char *msg,
                             const char *btn1, const char *btn2, const c
```

Popup avec 3 boutons. Retourne `1`, `2` ou `3`.

```
int show_msg_user_input(const char *title, const char *msg,
                        char *defaultvalue, char **value_ref);
```

Popup avec champ texte. Retourne la longueur du texte entré, ou `-1` si annulé/vide.

```
// Exemple
char *valeur;
int len = show_msg_user_input("Nom", "Entre ton pseudo :", "Player1", &va
if (len > 0) {
    show_msgbox("Bonjour", valeur);
    free(valeur);  // IMPORTANT : toujours free() après utilisation
}
```

```
int show_1numeric_input(const char *title, const char *subtitle,
                        const char *msg, int *value_ref,
                        int min_value, int max_value);
```

Popup pour saisir un nombre. Retourne `1` si OK, `0` si annulé.

⚠️ `min_value` ne doit pas être `0` ou `-1` (annule la popup).

```
// Exemple
int vitesse = 5;
show_1numeric_input("Config", "Vitesse", "Valeur (1-10) :", &vitesse, 1,
```

```
void refresh_osscr(void);
```

À appeler à la fin du programme s'il a créé ou supprimé des fichiers, pour rafraîchir le navigateur de l'OS.

4.3 Clavier

```
BOOL any_key_pressed(void);
```

Retourne `TRUE` si n'importe quelle touche est pressée (non-bloquant).

```
BOOL isKeyPressed(key);
```

Teste une touche précise (non-bloquant). `key` = constante `KEY_NSPIRE_*`.

```
void wait_key_pressed(void);
```

Bloque jusqu'à ce qu'une touche soit pressée.

```
void wait_no_key_pressed(void);
```

Bloque jusqu'à ce que toutes les touches soient relâchées.

```
BOOL on_key_pressed(void);
```

Teste si la touche ON est pressée.

Constantes KEY_NSPIRE — touches principales :

Constante	Touche
`KEY_NSPIRE_UP`	Flèche haut
`KEY_NSPIRE_DOWN`	Flèche bas
`KEY_NSPIRE_LEFT`	Flèche gauche
`KEY_NSPIRE_RIGHT`	Flèche droite
`KEY_NSPIRE_ENTER`	Entrée

Constante	Touche
`KEY_NSPIRE_ESC`	Échap
`KEY_NSPIRE_DEL`	Suppr
`KEY_NSPIRE_SPACE`	Espace
`KEY_NSPIRE_0` … `KEY_NSPIRE_9`	Chiffres 0–9
`KEY_NSPIRE_A` … `KEY_NSPIRE_Z`	Lettres A–Z
`KEY_NSPIRE_PERIOD`	Point
`KEY_NSPIRE_COMMA`	Virgule
`KEY_NSPIRE_PLUS`	+
`KEY_NSPIRE_MINUS`	–
`KEY_NSPIRE_MULT`	×
`KEY_NSPIRE_DIV`	÷
`KEY_NSPIRE_CTRL`	Ctrl
`KEY_NSPIRE_SHIFT`	Shift
`KEY_NSPIRE_BAR`	Barre (menu)
`KEY_NSPIRE_CLICK`	Clic du pavé tactile
`KEY_NSPIRE_SCRATCHPAD`	Scratchpad

Exemple — boucle de jeu avec input :

```
while (1) {
    if (isKeyPressed(KEY_NSPIRE_ESC)) break;
    if (isKeyPressed(KEY_NSPIRE_UP))    y--;
    if (isKeyPressed(KEY_NSPIRE_DOWN))  y++;
    if (isKeyPressed(KEY_NSPIRE_LEFT))  x--;
    if (isKeyPressed(KEY_NSPIRE_RIGHT)) x++;

    // Dessiner...
    idle();  // Économise la batterie entre les frames
}
```

4.4 Temps & CPU

```
void msleep(unsigned ms);
```

Pause en millisecondes.

```
void idle(void);
```

Met le CPU en veille basse consommation jusqu'à la prochaine interruption. À utiliser dans les boucles d'attente pour économiser la batterie.

```
void clear_cache(void);
```

Vide le cache d'instructions et de données. Utile si tu modifies du code dynamiquement.

4.5 Filesystem

```
int enable_relative_paths(char **argv);
```

Active les chemins relatifs pour `fopen()` etc. Passe `argv` de `main`. Retourne `0` si OK, `-1` si erreur.

```
// main avec argv pour chemins relatifs
int main(int argc, char *argv[]) {
    enable_relative_paths(argv);
    FILE *f = fopen("data.txt", "r");
    // ...
}
```

4.6 Détection hardware

```
BOOL is_classic;     // TRUE sur vieilles Nspire non-CX
BOOL has_colors;     // TRUE sur CX (toujours TRUE sur ta calc)
BOOL is_cm;          // TRUE sur CM/CM-C
BOOL is_touchpad;    // TRUE sur Touchpad et CX
unsigned hwtype();   // 0 = classic, 1 = CX
```

```
// Pattern recommandé pour code multi-modèle
if (is_classic) {
    // Code grayscale
} else {
    // Code couleur CX
}
```

4.7 Touchpad

```
touchpad_info_t *touchpad_getinfo(void);
```

Infos sur le pavé tactile (dimensions, etc.). Retourne `NULL` si pas de touchpad.

```
int touchpad_scan(touchpad_report_t *report);
```

Lit l'état du touchpad. `report->contact` = doigt posé, `report->pressed` = clic.

```
// Exemple
touchpad_report_t tpad;
touchpad_scan(&tpad);
if (tpad.contact) {
    // Un doigt est sur le pavé
    int tx = tpad.x;
    int ty = tpad.y;
}
```

4.8 Divers

```
void assert_ndless_rev(unsigned required_rev);
```

Affiche une popup et quitte si la révision Ndless est trop ancienne. À mettre au début de `main` si ton programme nécessite une version précise.

```
// Exemple
assert_ndless_rev(1000);  // Nécessite Ndless r1000 minimum
```

```
const char *NDLESS_DIR;
```

Chemin complet du dossier ndless : `"/documents/ndless"` .

---

5. os.h — libc standard

Header : `#include <os.h>`
Donne accès à la libc standard (newlib) + fonctions POSIX.

Fonctions disponibles (sélection utile)

Mémoire :

```
void *malloc(size_t size);
void free(void *ptr);
void *calloc(size_t n, size_t size);   // malloc + zéro-initialise
void *realloc(void *ptr, size_t size);
void *memset(void *s, int c, size_t n);
void *memcpy(void *dst, const void *src, size_t n);
int   memcmp(const void *a, const void *b, size_t n);
```

Strings :

```
size_t strlen(const char *s);
char  *strcpy(char *dst, const char *src);
char  *strncpy(char *dst, const char *src, size_t n);
char  *strcat(char *dst, const char *src);
int    strcmp(const char *a, const char *b);
int    strncmp(const char *a, const char *b, size_t n);
char  *strchr(const char *s, int c);
char  *strstr(const char *haystack, const char *needle);
int    sprintf(char *buf, const char *fmt, ...);
int    snprintf(char *buf, size_t n, const char *fmt, ...);
int    sscanf(const char *str, const char *fmt, ...);
```

Fichiers :

```
FILE *fopen(const char *path, const char *mode);
int   fclose(FILE *f);
```

```
int   fread(void *buf, size_t size, size_t n, FILE *f);
int   fwrite(const void *buf, size_t size, size_t n, FILE *f);
int   fprintf(FILE *f, const char *fmt, ...);
int   fscanf(FILE *f, const char *fmt, ...);
int   fseek(FILE *f, long offset, int whence);
long  ftell(FILE *f);
int   feof(FILE *f);
int   remove(const char *path);
int   rename(const char *old, const char *new);
```

Modes fopen :

Mode	Description
`"r"`	Lecture
`"w"`	Écriture (crée/écrase)
`"a"`	Append
`"rb"`	Lecture binaire
`"wb"`	Écriture binaire

Maths :

```
#include <math.h>

double sin(double x);
double cos(double x);
double tan(double x);
double sqrt(double x);
double pow(double base, double exp);
double fabs(double x);
double floor(double x);
double ceil(double x);
double fmod(double x, double y);
```

Utilitaires :

```
int    abs(int x);
int    rand(void);          // Nombre aléatoire [0, RAND_MAX]
void   srand(unsigned seed);
void   qsort(void *base, size_t n, size_t size,
             int (*cmp)(const void*, const void*));
```

Exemple — lire/écrire un fichier :

```
#include <libndls.h>
#include <os.h>

int main(int argc, char *argv[]) {
    enable_relative_paths(argv);

    // Écrire
    FILE *f = fopen("score.txt", "w");
    if (f) {
        fprintf(f, "1500\n");
        fclose(f);
    }

    // Lire
    int score = 0;
    f = fopen("score.txt", "r");
    if (f) {
        fscanf(f, "%d", &score);
        fclose(f);
    }

    char msg[64];
    sprintf(msg, "Score : %d", score);
    show_msgbox("Résultat", msg);

    return 0;
}
```

---

6. nGC — API graphique TI-Nspire

Header : `#include <ngc.h>`

API graphique native de l'OS, similaire à Java Graphics2D. Travaille sur un buffer off-screen, puis blit à l'écran.

6.1 Initialisation

```
// Récupérer le contexte graphique global de l'OS
Gc gc = *gui_gc_global_GC_ptr;
```

```
// Pattern recommandé (avec setRegion pour que clipRect fonctionne)
gui_gc_setRegion(gc, 0, 0, 320, 240, 0, 0, 320, 240);
gui_gc_begin(gc);

// ... dessiner ...

gui_gc_blit_to_screen(gc);  // Envoyer à l'écran
gui_gc_finish(gc);           // Libérer les ressources internes
```

> Le gc est un buffer **off-screen**. Rien n'est visible tant que tu n'as pas appelé `gui_gc_blit_to_screen`.

6.2 Couleur & stylo

```
void gui_gc_setColorRGB(Gc gc, int r, int g, int b);
```

Définit la couleur active. R, G, B dans [0..255].

```
void gui_gc_setColor(Gc gc, int color);
```

Même chose mais avec format `0xRRGGBB`.

```
void gui_gc_setPen(Gc gc, gui_gc_PenSize size, gui_gc_PenMode mode);
```

Taille et mode du stylo.

PenSize	Description
`GC_PS_THIN`	Fin
`GC_PS_MEDIUM`	Moyen
`GC_PS_THICK`	Épais

PenMode	Description
`GC_PM_NORMAL`	Normal
`GC_PM_DOTTED`	Pointillés

6.3 Formes

```
void gui_gc_fillRect(Gc gc, int x, int y, int w, int h);
```

Rectangle plein.

```
void gui_gc_drawRect(Gc gc, int x, int y, int w, int h);
```

Rectangle vide (contour seulement).

```
void gui_gc_fillArc(Gc gc, int x, int y, int w, int h, int start, int enc
```

Arc/cercle plein. `start` et `end` en degrés × 10 (donc 0 à 3600 pour un cercle complet).

```
// Cercle plein centré en (100,100), rayon 20
gui_gc_fillArc(gc, 80, 80, 40, 40, 0, 3600);
```

```
void gui_gc_drawArc(Gc gc, int x, int y, int w, int h, int start, int enc
```

Arc/cercle vide.

```
void gui_gc_drawLine(Gc gc, int x1, int y1, int x2, int y2);
```

Ligne.

```
void gui_gc_fillPoly(Gc gc, int *points, int count);
void gui_gc_drawPoly(Gc gc, int *points, int count);
```

Polygone. `points` = tableau de coordonnées x,y alternées. `count` = nombre total de valeurs.

```
// Triangle
int pts[] = {160,20, 280,220, 40,220, 160,20};
gui_gc_fillPoly(gc, pts, 8);
```

```
void gui_gc_fillGradient(Gc gc, int x, int y, int w, int h,
                         int start_color, int end_color, int vertical);
```

Dégradé. `start_color` et `end_color` en `0xRRGGBB`. `vertical=1` pour vertical, `0` pour horizontal.

6.4 Texte

```
void gui_gc_drawString(Gc gc, char *utf16, int x, int y, gui_gc_StringMoc
```

Affiche une chaîne UTF-16. Le texte en C normal est ASCII/UTF-8, il faut le convertir.

```
void gui_gc_setFont(Gc gc, gui_gc_Font font);
```

Font	Description
GC_FONT_SMALL	Petite
GC_FONT_NORMAL	Normale
GC_FONT_BOLD	Gras

StringMode	Description
GC_SM_TOP	y = bord haut du texte
GC_SM_MIDDLE	y = milieu du texte
GC_SM_DOWN	Texte à l'envers

```
// Afficher "Hello" — notation UTF-16 inline
gui_gc_drawString(gc, "H\0e\0l\0l\0o\0\0", 50, 50, GC_SM_TOP);
```

Convertir ASCII → UTF-16 proprement :

```
void ascii2utf16(void *buf, const char *str, int max_size);

// Exemple
char utf16[64];
ascii2utf16(utf16, "Score: 100", sizeof(utf16));
gui_gc_drawString(gc, utf16, 10, 10, GC_SM_TOP);
```

Mesurer la taille du texte :

```
int gui_gc_getStringWidth(Gc gc, gui_gc_Font font, char *utf16, int start
int gui_gc_getStringHeight(Gc gc, gui_gc_Font font, char *utf16, int star
int gui_gc_getFontHeight(Gc gc, gui_gc_Font font);
```

6.5 Clipping

```
void gui_gc_clipRect(Gc gc, int x, int y, int w, int h, gui_gc_ClipRectOp
```

Limite le dessin à une zone.

ClipRectOp	Description
`GC_CRO_SET`	Définit la zone de clip
`GC_CRO_RESET`	Supprime le clip (plein écran)

6.6 Blit

```
void gui_gc_blit_to_screen(Gc gc);
```

Envoie tout le buffer à l'écran.

```
void gui_gc_blit_to_screen_region(Gc gc, int x, int y, int w, int h);
```

Envoie seulement une région (plus rapide pour les mises à jour partielles).

```
void gui_gc_blit_gc(Gc src, int xs, int ys, int ws, int hs,
                    Gc dst, int xd, int yd, int wd, int hd);
```

Copie (et étire si nécessaire) d'un gc vers un autre.

6.7 Exemple complet nGC

```
#include <libndls.h>
#include <os.h>
#include <ngc.h>

int main(void) {
    Gc gc = *gui_gc_global_GC_ptr;
    gui_gc_setRegion(gc, 0, 0, 320, 240, 0, 0, 320, 240);
    gui_gc_begin(gc);

    // Fond noir
    gui_gc_setColorRGB(gc, 0, 0, 0);
```

```
    gui_gc_fillRect(gc, 0, 0, 320, 240);

    // Carré rouge
    gui_gc_setColorRGB(gc, 255, 0, 0);
    gui_gc_fillRect(gc, 100, 80, 120, 80);

    // Cercle bleu
    gui_gc_setColorRGB(gc, 0, 100, 255);
    gui_gc_fillArc(gc, 140, 100, 40, 40, 0, 3600);

    // Texte blanc
    char utf16[32];
    ascii2utf16(utf16, "Hello!", sizeof(utf16));
    gui_gc_setColorRGB(gc, 255, 255, 255);
    gui_gc_setFont(gc, GC_FONT_BOLD);
    gui_gc_drawString(gc, utf16, 120, 10, GC_SM_TOP);

    gui_gc_blit_to_screen(gc);
    gui_gc_finish(gc);

    while (!any_key_pressed()) {}
    return 0;
}
```

---

7. SDL 1.2

Header : `#include <SDL/SDL.h>`

Makefile : `LIBS = -lSDL`

SDL est la lib la plus complète pour les jeux. Elle gère la fenêtre, les events, le rendu de surfaces, et plus.

7.1 Initialisation

```
SDL_Init(SDL_INIT_VIDEO);
SDL_Surface *screen = SDL_SetVideoMode(320, 240, 16, SDL_SWSURFACE);
SDL_ShowCursor(SDL_DISABLE);
```

7.2 Couleurs

```
Uint32 couleur = SDL_MapRGB(screen->format, r, g, b);
// r, g, b dans [0..255]
```

7.3 Dessiner

```
// Rectangle plein
SDL_Rect rect = {x, y, w, h};
SDL_FillRect(screen, &rect, couleur);

// Remplir tout l'écran
SDL_FillRect(screen, NULL, couleur);
```

7.4 Afficher (flip)

```
SDL_Flip(screen);  // ou SDL_UpdateRect(screen, 0, 0, 0, 0);
```

7.5 Events & clavier

```
SDL_Event event;
while (SDL_PollEvent(&event)) {
    if (event.type == SDL_KEYDOWN) {
        switch (event.key.keysym.sym) {
            case SDLK_UP:    /* flèche haut */ break;
            case SDLK_DOWN:  /* flèche bas  */ break;
            case SDLK_LEFT:  /* flèche gauche */ break;
            case SDLK_RIGHT: /* flèche droite  */ break;
            case SDLK_RETURN: /* entrée */ break;
            case SDLK_ESCAPE: /* échap  */ break;
        }
    }
    if (event.type == SDL_QUIT) quitter = 1;
}
```

Touches SDL sur Nspire :

SDLK	Touche calc
`SDLK_UP`	Flèche haut

SDLK	Touche calc
SDLK_DOWN	Flèche bas
SDLK_LEFT	Flèche gauche
SDLK_RIGHT	Flèche droite
SDLK_RETURN	Entrée
SDLK_ESCAPE	Échap
SDLK_SPACE	Espace
SDLK_BACKSPACE	Suppr
SDLK_a … SDLK_z	Lettres
SDLK_0 … SDLK_9	Chiffres

7.6 Surfaces & images

```
// Créer une surface
SDL_Surface *surf = SDL_CreateRGBSurface(SDL_SWSURFACE, w, h, 16, 0, 0, 0

// Copier sur l'écran (blit)
SDL_Rect src = {0, 0, w, h};
SDL_Rect dst = {x, y, 0, 0};
SDL_BlitSurface(surf, &src, screen, &dst);

// Libérer
SDL_FreeSurface(surf);
```

7.7 Accès direct aux pixels (SDL)

```
SDL_LockSurface(screen);
Uint16 *pixels = (Uint16*)screen->pixels;
pixels[y * (screen->pitch / 2) + x] = couleur;
SDL_UnlockSurface(screen);
```

7.8 Timing

```
Uint32 ticks = SDL_GetTicks();  // Millisecondes depuis SDL_Init
SDL_Delay(ms);                  // Pause en ms
```

7.9 Quitter SDL

```
SDL_Quit();
```

7.10 Exemple complet SDL

```
#include <libndls.h>
#include <SDL/SDL.h>

int main(void) {
    SDL_Init(SDL_INIT_VIDEO);
    SDL_Surface *screen = SDL_SetVideoMode(320, 240, 16, SDL_SWSURFACE);
    SDL_ShowCursor(SDL_DISABLE);

    int x = 150, y = 110;
    int running = 1;

    while (running) {
        SDL_Event e;
        while (SDL_PollEvent(&e)) {
            if (e.type == SDL_KEYDOWN) {
                if (e.key.keysym.sym == SDLK_ESCAPE) running = 0;
                if (e.key.keysym.sym == SDLK_UP)    y -= 2;
                if (e.key.keysym.sym == SDLK_DOWN)  y += 2;
                if (e.key.keysym.sym == SDLK_LEFT)  x -= 2;
                if (e.key.keysym.sym == SDLK_RIGHT) x += 2;
            }
        }

        // Fond bleu foncé
        SDL_FillRect(screen, NULL, SDL_MapRGB(screen->format, 10, 10, 40)

        // Carré blanc (joueur)
        SDL_Rect rect = {x, y, 20, 20};
        SDL_FillRect(screen, &rect, SDL_MapRGB(screen->format, 255, 255,

        SDL_Flip(screen);
        SDL_Delay(16);  // ~60 FPS
    }
```

```
    SDL_Quit();
    return 0;
}
```

---

8. nGL

Header : `#include <nGL.h>` (ou selon installation)

Makefile : `LIBS = -lngl`

nGL est un moteur graphique 2D/3D software pour Nspire. Il est particulièrement adapté pour les jeux nécessitant des transformations, sprites, ou rendu 3D basique.

> nGL est plus bas niveau que SDL pour la 2D, mais offre des fonctionnalités 3D que SDL n't a pas.

8.1 Initialisation

```
nglInit();
```

8.2 Rendu

```
nglDisplay();  // Affiche le buffer courant à l'écran
```

8.3 Pixels

```
// Écrire un pixel (coordonnées, couleur RGB565)
nglSetPixel(x, y, color);

// Lire un pixel
uint16_t color = nglGetPixel(x, y);
```

8.4 Couleurs nGL (RGB565)

```
// nGL utilise RGB565 directement
#define NGL_COLOR(r, g, b) (((r >> 3) << 11) | ((g >> 2) << 5) | (b >> 3)

uint16_t rouge = NGL_COLOR(255, 0, 0);
```

```
uint16_t vert  = NGL_COLOR(0, 255, 0);
uint16_t bleu  = NGL_COLOR(0, 0, 255);
```

8.5 Cleanup

```
nglUninit();
```

---

9. FreeType

Header : `#include <ft2build.h>` et `#include FT_FREETYPE_H`

Makefile : `LIBS = -lfreetype`

FreeType permet de rendre des polices TTF avec anticrénelage. C'est la lib à utiliser si tu veux un rendu texte de qualité avec tes propres polices.

9.1 Initialisation

```
FT_Library ft;
FT_Init_FreeType(&ft);

FT_Face face;
FT_New_Face(ft, "/documents/ndless/mafonte.ttf", 0, &face);
FT_Set_Pixel_Sizes(face, 0, 16);  // Taille en pixels
```

9.2 Rendre un caractère

```
FT_Load_Char(face, 'A', FT_LOAD_RENDER);
FT_Bitmap *bitmap = &face->glyph->bitmap;
// bitmap->buffer contient les données grises [0..255]
// bitmap->width, bitmap->rows = dimensions
```

9.3 Cleanup

```
FT_Done_Face(face);
FT_Done_FreeType(ft);
```

> FreeType génère un bitmap grayscale. Tu dois le copier manuellement dans ton framebuffer (SDL ou direct) en blendant avec la couleur de fond.

---

10. zlib

Header : `#include <zlib.h>`

Makefile : `LIBS = -lz`

zlib compresse et décompresse des données. Utile pour charger des assets compressés ou sauvegarder de la data.

10.1 Compresser

```
uLong src_len = strlen(data) + 1;
uLong dst_len = compressBound(src_len);
Bytef *dst = malloc(dst_len);

int ret = compress(dst, &dst_len, (Bytef*)data, src_len);
// ret == Z_OK si succès
// dst_len est mis à jour avec la taille compressée réelle
```

10.2 Décompresser

```
uLong dst_len = taille_originale_connue;
Bytef *dst = malloc(dst_len);

int ret = uncompress(dst, &dst_len, src, src_len);
// ret == Z_OK si succès
```

10.3 Niveaux de compression

```
// compress2 permet de choisir le niveau (0=aucun, 9=max)
compress2(dst, &dst_len, src, src_len, Z_BEST_COMPRESSION);
compress2(dst, &dst_len, src, src_len, Z_BEST_SPEED);
compress2(dst, &dst_len, src, src_len, Z_DEFAULT_COMPRESSION);  // niveau
```

---

11. Patterns & techniques utiles

11.1 Boucle de jeu basique

```
#include <libndls.h>
#include <os.h>
#include <SDL/SDL.h>

#define FPS 60
#define FRAME_MS (1000 / FPS)

int main(void) {
    SDL_Init(SDL_INIT_VIDEO);
    SDL_Surface *screen = SDL_SetVideoMode(320, 240, 16, SDL_SWSURFACE);

    int running = 1;
    Uint32 last = SDL_GetTicks();

    while (running) {
        Uint32 now = SDL_GetTicks();
        float dt = (now - last) / 1000.0f;  // Delta time en secondes
        last = now;

        // Input
        SDL_Event e;
        while (SDL_PollEvent(&e))
            if (e.type == SDL_KEYDOWN && e.key.keysym.sym == SDLK_ESCAPE)
                running = 0;

        // Update (utilise dt pour vitesse indépendante du FPS)
        // x += vitesse * dt;

        // Rendu
        SDL_FillRect(screen, NULL, 0);
        // ... dessiner ...
        SDL_Flip(screen);

        // Cap FPS
        Uint32 elapsed = SDL_GetTicks() - now;
        if (elapsed < FRAME_MS)
            SDL_Delay(FRAME_MS - elapsed);
    }
```

```
    SDL_Quit();
    return 0;
}
```

11.2 Afficher un nombre à l'écran (nGC)

```
char buf[32];
char utf16[64];
sprintf(buf, "Score: %d", score);
ascii2utf16(utf16, buf, sizeof(utf16));
gui_gc_setColorRGB(gc, 255, 255, 255);
gui_gc_drawString(gc, utf16, x, y, GC_SM_TOP);
```

11.3 Double buffering (framebuffer direct)

```
uint16_t buf_a[320 * 240];
uint16_t buf_b[320 * 240];
uint16_t *back  = buf_a;
uint16_t *front = buf_b;

// Dans la boucle :
// Dessiner dans back...
lcd_blit(back, SCR_320x240_565);
// Swap
uint16_t *tmp = back; back = front; front = tmp;
```

11.4 Détecter un appui (pas un maintien)

```
int was_up = 0;

// Dans la boucle :
int is_up = isKeyPressed(KEY_NSPIRE_UP);
if (is_up && !was_up) {
    // Vient d'être pressé
}
was_up = is_up;
```

11.5 Clamp (borner une valeur)

```
// Pas de clamp dans la libc standard — définis-le toi-même
#define CLAMP(v, lo, hi) ((v) < (lo) ? (lo) : (v) > (hi) ? (hi) : (v))

x = CLAMP(x, 0, 319);
y = CLAMP(y, 0, 239);
```

11.6 Chemins de fichiers sur la calc

Les programmes Ndless tournent depuis `/documents/` . Les chemins absolus recommandés :

```
/documents/                Racine des documents
/documents/ndless/         Dossier Ndless (NDLESS_DIR)
/documents/ndless/monprog.tns
```

Avec `enable_relative_paths(argv)` , les chemins relatifs partent du dossier du `.tns` .

---

12. Erreurs fréquentes

Erreur	Cause	Fix
Freeze + "compatibility mode"	`--240x320-support false`	Mettre `true`
Écran blanc/noir, pas de rendu nGC	Oubli de `gui_gc_blit_to_screen`	Ajouter le blit
Crash au lancement	`lcd_init` pas appelé avant `lcd_blit`	Appeler `lcd_init` d'abord
Texte nGC illisible	Chaîne pas convertie en UTF-16	Utiliser `ascii2utf16`
Fuite mémoire	`malloc` sans `free`	Toujours `free` après usage
`fopen` retourne NULL	Chemins relatifs non activés	Appeler `enable_relative_paths(argv)`
Warning `clock skew` dans make	Heure du container Docker décalée	Inoffensif, ignorer
Warning `lcd_blit API` dans genzehn	Pas d'API blit détectée	Inoffensif si tu utilises nGC ou SDL

---

13. Cheatsheet — includes & LIBS

Lib	Include	LIBS dans Makefile
libndls	`#include <libndls.h>`	*(auto)*
libc / os	`#include <os.h>`	*(auto)*
nGC	`#include <ngc.h>`	*(auto, dans os.h)*
SDL 1.2	`#include <SDL/SDL.h>`	`-lSDL`
nGL	`#include <nGL.h>`	`-lngl`
FreeType	`#include <ft2build.h>` + `#include FT_FREETYPE_H`	`-lfreetype`
zlib	`#include <zlib.h>`	`-lz`
math	`#include <math.h>`	`-lm`

Guide généré pour Ndless SDK — TI-Nspire CX CAS II-T (HW-W) — 2026